

instrumentation, observability, or real-time examples, missing fundamental or even compliant medical advice.

Vague Security and Compliance Specifications (OWASP/HIPAA): The architecture handles highly sensitive medical audio and PII but lacks explicit API Gateway security controls (rate limiting, input validation, injection protection) and specific encryption standards (AES-256, TLS 1.3).

WHAT INDUSTRY EXPECTS

In a mission-critical medical emergency domain, the industry expects an enterprise-grade AI system to prioritize absolute reliability, safety, and strict regulatory compliance. This requires full compliance with HIPAA and GDPR for patient data protection, including end-to-end encryption (AES-256 at rest, TLS 1.3 in transit) and robust API security following OWASP Top 10 guidelines. Architecturally, services must adhere to the 12-Factor App methodology to ensure statelessness, horizontal scalability, and clean configuration boundaries. Furthermore, Responsible AI principles demand complete auditability and explainability of all automated triage decisions, backed by comprehensive LLM observability (OpenTelemetry) to trace latency, token usage, and prompt performance in real-time.

WHAT WAS DELIVERED

The delivered submission provides a highly detailed and well-thought-out architectural blueprint that successfully integrates the mandatory hackathon stack (Moss, LiveKit, Next.js) and establishes a solid multi-party WebRTC communication flow. It introduces excellent domain-specific features such as prosody analysis, a persistent Redis-backed session cache, and an automated SOAP note generator. However, it falls short of enterprise-grade standards by completely omitting LLM observability and tracing frameworks like OpenTelemetry. Additionally, the PRD lacks a rigorous prompt engineering specification (such as CRISPE), fails to define unique IDs or quantitative acceptance criteria for its requirements, and provides only high-level security guidelines without detailing OWASP API protections or specific encryption protocols.

RECOMMENDATIONS

High Technical Depth

Complete absence of LLM observability and tracing (OpenTelemetry) for tracking latency, token usage, and prompt performance in a life-critical system.

Integrate OpenTelemetry auto-instrumentation for LangGraph and Pydantic AI. Explicitly document how you will log token usage, latency, prompt hashes, and model versions to an observability platform like Arize Phoenix or LangSmith, using correlation IDs across the entire system boundaries.



Mentor Agent



High Security & Compliance